

Noteworthy

轻量化在线笔记应用 — 项目说明文档

前后端分离 • 富文本编辑 • JWT 认证 • 响应式设计

姓名：XXX 学号：XXXXXXXXXX

选题方向：Web 全栈应用开发

2026 年 6 月 11 日

目录

1	项目基本信息	3
1.1	运行方式	3
2	产品概述	3
2.1	产品定位	3
2.2	功能全景	4
3	技术架构	4
3.1	技术选型与决策	4
3.2	系统架构	4
3.3	数据模型	5
4	核心功能实现	5
4.1	JWT 双令牌认证	5
4.2	富文本编辑器	6
4.3	图片生命周期管理	6
4.4	全文搜索	6
5	技术挑战与解决方案	6
6	项目结构与 API	7
7	学习总结与自我评估	8
7.1	学习收获	8
7.2	完成度评估	8

1 项目基本信息

- **项目名称：**Noteworthy — 轻量化在线笔记软件
- **选题方向：**Web 全栈应用开发
- **作者：**XXX **学号：**XXXXXXXXX

1.1 运行方式

前置依赖： Docker & Docker Compose、Python 3.10+、Node.js 18+

1. **启动数据库：** `docker compose up -d db`

2. **启动后端：**

```
cd backend && cp .env.example .env
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt && alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

访问 <http://localhost:8000/docs> 查看 Swagger API 文档。

3. **启动前端：**

```
cd frontend && npm install && npm run dev
```

浏览器打开 <http://localhost:5173> 即可使用。

2 产品概述

2.1 产品定位

Noteworthy 定位为**轻量化、高可用的在线笔记平台**，面向个人知识管理场景，提供从采集、组织到检索的完整笔记工作流。与同类产品相比，其核心差异化优势在于：

- **全栈自研：**前后端完全自主开发，无第三方笔记服务依赖，数据主权可控。
- **专业编辑体验：**基于 ProseMirror 内核的 Tiptap 编辑器，支持 15+ 富文本扩展、代码语法高亮与图片上传，编辑能力远超 Markdown 纯文本方案。
- **企业级安全：**BCrypt(12 rounds) 密码哈希 + JWT 双令牌无感续期 + 用户级数据隔离，安全机制对标生产环境标准。
- **智能资源管理：**双层图片清理策略（即时 + 定时兜底）与归档笔记自动过期，解决服务器存储泄漏问题。

功能域	能力描述
用户认证	注册/登录、JWT 双令牌鉴权、令牌自动刷新、导航守卫
笔记本管理	创建/重命名/删除、自定义图标与颜色、拖拽排序、归档
笔记编辑	Tiptap 富文本 (15+ 扩展)、3s 防抖自动保存、Ctrl+S 手动保存、字数统计
笔记组织	置顶、归档/恢复、标签多对多关联、跨笔记本聚合视图
图片系统	上传 (≤5 MB, 5 种格式)、即时孤立清理 + 定时全量扫描、归档 >7 天自动删除
全文搜索	标题 + 纯文本 ILIKE 模糊匹配、按笔记本过滤、分页
UI/UX	玻璃拟态设计、深色模式、响应式三端适配、骨架屏 + Toast + 页面过渡动画

2.2 功能全景

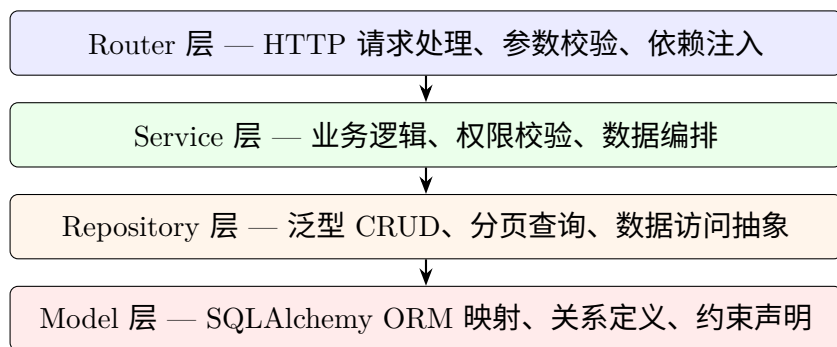
3 技术架构

3.1 技术选型与决策

层面	技术	选型理由
前端框架	Vue 3 + TS	Composition API + <code><script setup></code> 简洁高效; TypeScript 全量类型覆盖
构建工具	Vite 6	原生 ESM HMR, 冷启动 <300 ms, 较 Webpack 提升 10–100×
状态管理	Pinia	Vue 3 官方推荐, Setup Stores 与 Composition API 风格统一, TS 推导完善
样式方案	UnoCSS	原子化按需生成, 零运行时开销; 兼容 Tailwind 生态
富文本	Tiptap	ProseMirror 内核, 可扩展架构支持表格/代码高亮/任务列表等企业级功能
后端框架	FastAPI	async/await 原生、自动 OpenAPI 文档、Pydantic v2 校验性能提升 5–50×
ORM	SQLAlchemy 2.0	声明式映射 + <code>select()</code> 风格 + 异步引擎, 2.0 重写后类型提示完备
数据库	PostgreSQL 16	JSONB 原生存储 Tiptap 文档、pg_trgm 三字符索引加速模糊搜索
认证	JWT 双令牌	Access(30 min) + Refresh(7 d), 无状态鉴权, 水平扩展友好
容器化	Docker Compose	postgres:16-alpine 镜像, healthcheck 就绪探针保障启动顺序

3.2 系统架构

后端采用**四层分离架构**, 严格遵循单一职责原则:



前端按功能域组织：API 模块封装接口调用与令牌刷新拦截器，Pinia Store 管理响应式状态，View 负责声明式渲染，UI 组件库提供一致的设计系统。

3.3 数据模型

系统共 5 张表，核心实体关系如下：

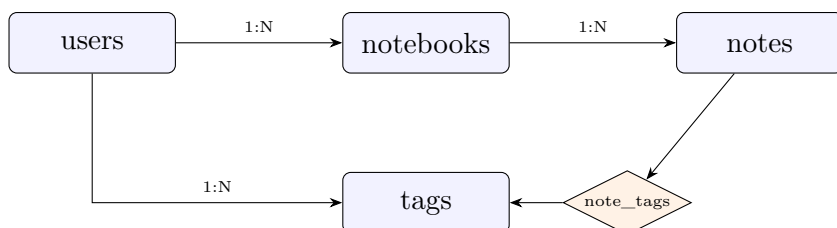


表	关键字段	设计要点
users	id(UUID), username, email, hashed_password	UUID 主键防枚举；username/email 唯一索引
notebooks	id, user_id(FK), name, icon, color, sort_order	CASCADE 级联删除笔记；sort_order 支持自定义排序
notes	id, notebook_id(FK), user_id(FK), content(JSONB), plain_text, is_pinned, is_archived	JSONB 存储 Tiptap 文档；plain_text 冗余字段加速搜索
tags	id, user_id(FK), name, color	(user_id, name) 联合唯一约束防重复
note_tags	note_id(PK), tag_id(PK)	多对多关联表，双主键 + CASCADE

4 核心功能实现

4.1 JWT 双令牌认证

1. **签发**：登录成功后签发 Access Token (30 min) 与 Refresh Token (7 d)，均使用 HS256 算法，payload 含 sub/iat/exp/type 声明。

2. **鉴权**: 请求经 `get_current_user` 依赖注入解析 Access Token, 校验 `type="access"` 及有效期, 注入用户对象至路由函数。
3. **无感续期**: 前端 Axios 响应拦截器捕获 401 后, 以 Refresh Token 调用 `/auth/refresh` 获取新令牌对。**并发控制**: 首个 401 触发刷新, 其余请求入队等待, 刷新完成后统一重试, 避免竞态导致 Token 失效。
4. **数据隔离**: 所有 Service 层查询均携带 `user_id` 条件, 确保用户间数据物理隔离。

4.2 富文本编辑器

基于 Tiptap (ProseMirror 内核) 构建, 集成 15+ 扩展:

- StarterKit (段落/标题/列表/引用/代码)
- TextStyle + Color (16 色预设 + 拾色器)
- CodeBlockLowlight (lowlight 语法高亮)
- Highlight / Underline / Link
- Table + TableRow/Cell/Header
- Typography / Placeholder
- TaskList + TaskItem (嵌套)
- CharacterCount (实时字数统计)
- Image (服务端上传, 非 Base64)
- 自定义 CodeBlockWithLabel (语言标签)

保存策略: 编辑器 `onUpdate` 触发 3s 防抖定时器, 标题变更同样触发; `Ctrl+S` 立即保存; 组件卸载时若 `isDirty` 则兜底保存。状态栏三态指示: **Saved** / **Unsaved** / **Saving...**。

4.3 图片生命周期管理

1. **上传**: `POST /uploads/images`, 校验 Content-Type (JPEG/PNG/GIF/WebP/SVG) 与大小 (≤ 5 MB), UUID 重命名存储至 `uploads/`, 返回 `/uploads/<filename> URL`。
2. **即时清理**: 笔记更新时 diff 新旧 content 中的图片引用集合, 删除不再被引用的文件; 笔记删除时同步清理关联图片。
3. **定时兜底**: 后台 `cleanup_loop` 每小时全量扫描: (a) 删除 `uploads/` 中未被任何 note 引用的文件; (b) 删除 `archived_at` 超过 7 天的归档笔记及其标签关联。

4.4 全文搜索

笔记创建/更新时, 后端递归遍历 Tiptap JSON 提取 `type="text"` 节点, 拼接为 `plain_text` 冗余字段。搜索接口基于 ILIKE 对 `title` 和 `plain_text` 进行模式匹配, 支持按 `notebook_id` 过滤与分页。数据库初始化时已启用 `pg_trgm` 扩展, 可平滑升级为三字符索引加速。

5 技术挑战与解决方案

挑战 1: JWT 并发刷新竞态。SPA 中多个异步请求同时收到 401 时, 若各自独立刷新令牌, 后到的刷新请求会使先前的 Refresh Token 失效。**方案**: 在 Axios 拦截器中实现请求队

列模式——设置 isRefreshing 互斥标志，首个 401 持有刷新权，其余以 Promise 入队；刷新成功后 processQueue(null, newToken) 统一重试，失败则全部拒绝并登出，O(1) 额外开销。

挑战 2：服务器图片存储泄漏。用户删除笔记或删除图片后，文件系统残留无人引用的图片。**方案：**双层清理——(1) **即时层：**NoteService.update/delete 中 diff 新旧 content 图片集合，立即删除孤立文件；(2) **兜底层：**cleanup_loop 每小时全量扫描，删除无主文件与归档超 7 天的笔记。两层互补确保零泄漏。

6 项目结构与 API

```
WebProgram/
+-- docker-compose.yml      # PostgreSQL 16 (alpine)
+-- docker/init.sql         # uuid-oss + pg_trgm
+-- backend/app/
| +-- main.py               # FastAPI app factory & lifespan
| +-- api/v1/               # Routers (auth, notebooks, notes, tags, search, uploads)
| +-- core/                 # config, security, database, exceptions
| +-- models/               # ORM (User, Notebook, Note, Tag, NoteTag)
| +-- schemas/              # Pydantic v2 schemas
| +-- services/             # Business logic + cleanup
| +-- repositories/         # Generic BaseRepository CRUD
+-- frontend/src/
| +-- api/                  # Axios client + refresh interceptor
| +-- components/{ui,layout} # Design system + layout
| +-- layouts/              # DefaultLayout, AuthLayout
| +-- router/               # Vue Router + authGuard
| +-- stores/               # Pinia (auth, notes, notebooks, tags, editor, ui)
| +-- views/                # 10 page views
+-- docs/                   # Project documentation
```

后端采用路由 → 服务 → 仓库 → 模型四层架构，前端按功能域组织。系统共暴露 16 个 API 端点：

方法	路径	说明
POST	/api/v1/auth/register	用户注册，返回双令牌
POST	/api/v1/auth/login	登录，BCrypt 校验
POST	/api/v1/auth/refresh	刷新令牌对
GET	/api/v1/auth/me	当前用户信息
GET/POST	/api/v1/notebooks	笔记本列表/创建
GET/PUT/DELETE	/api/v1/notebooks/{id}	笔记本 CRUD
GET/POST	/api/v1/notebooks/{id}/notes	笔记列表/创建
GET/PUT/DELETE	/api/v1/notes/{id}	笔记 CRUD
PATCH	/api/v1/notes/{id}/pin	置顶/取消
PATCH	/api/v1/notes/{id}/archive	归档/恢复
POST/DELETE	/api/v1/notes/{id}/tags	附加/移除标签
GET/POST	/api/v1/tags	标签列表/创建
PUT/DELETE	/api/v1/tags/{id}	标签更新/删除
GET	/api/v1/search?q= <i>kw</i>	全文搜索
POST	/api/v1/uploads/images	图片上传

7 学习总结与自我评估

7.1 学习收获

- 掌握前后端分离全栈开发完整流程：ER 设计、Alembic 迁移、RESTful API、SPA 交互。
- 深入理解 JWT 认证体系：双令牌策略、并发刷新队列、依赖注入鉴权、数据隔离。
- 实践 ProseMirror 文档模型：Tiptap 扩展机制、JSONB 存储、纯文本提取与搜索。
- 积累异步编程经验：SQLAlchemy 2.0 async、FastAPI async routes、asyncio 后台任务。
- 工程化能力提升：泛型 Repository、双层资源清理、Axios 拦截器设计。

7.2 完成度评估

全部 7 个功能模块均已实现（用户认证、笔记本 CRUD、笔记 CRUD、富文本编辑器、图片系统、搜索与标签、UI/UX），代码结构清晰，API 文档自动生成。

未来演进方向：tsvector 全文搜索 + GIN 索引替代 ILIKE；WebSocket 实时通知；笔记导出 Markdown/PDF；协作编辑（CRDT/OT）。

8 原创性声明与引用

本项目为个人独立完成的设计与开发作品，所有核心代码均为原创。

主要开源依赖：Vue 3 (<https://vuejs.org/>)、Pinia (<https://pinia.vuejs.org/>)、FastAPI (<https://fastapi.tiangolo.com/>)、SQLAlchemy (<https://www.sqlalchemy.org/>)、Tiptap (<https://tiptap.dev/>)、UnoCSS (<https://unocss.dev/>)、Phosphor Icons (<https://phosphoricons.com/>)、lowlight (<https://github.com/wooorm/lowlight>)